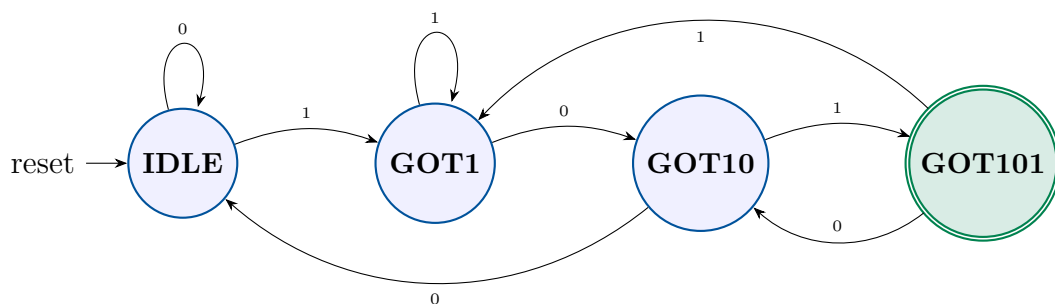


Programa de Apoio ao Desenvolvimento da
Indústria de Semicondutores
PADIS – Capacitação em Microeletrônica

Unidade 5 | Capítulo 2
Máquina de Estado Finita (FSM)
Detector de Sequência Serial “101”



Aluno: Matheus Serrão Uchôa – Matrícula: 22052633
Professor: Alexandre Almeida
Curso: Capacitação Profissional em Microeletrônica (PADIS)
Instituição: Universidade Federal do Amazonas (UFAM)
Data: Maio de 2026

Sumário

1	Introdução	2
2	Objetivos	2
3	Materiais e Ferramentas	2
3.1	Arquivos do Projeto	3
4	Fundamentos Teóricos	3
4.1	Máquinas de Estado Finitas (FSM)	3
4.2	Detecção da Sequência “101” com Sobreposição	3
4.3	Diagrama de Estados – Moore FSM (4 estados)	4
4.4	Diagrama de Estados – Mealy FSM (3 estados)	4
4.5	Tabelas de Transição de Estado	4
4.5.1	Moore FSM	5
4.5.2	Mealy FSM	5
4.6	Estilo Três Processos em SystemVerilog	5
5	Implementação em SystemVerilog	5
5.1	Moore FSM – <code>sequence_detector_moore.sv</code>	5
5.2	Mealy FSM – <code>sequence_detector_mealy.sv</code>	7
6	Testbenches	9
6.1	Sequência de Verificação	9
6.2	Testbench Moore – <code>tb_moore.sv</code>	10
6.3	Testbench Mealy – <code>tb_mealy.sv</code>	13
7	Resultados de Simulação	16
7.1	Compilação e Execução	16
7.2	Saída do Console – Moore FSM	16
7.3	Saída do Console – Mealy FSM	17
7.4	Formas de Onda – Moore FSM	17
7.5	Formas de Onda – Mealy FSM	18
7.6	Comparação das Saídas	18
8	Análise Comparativa	18
8.1	Número de Estados e Complexidade	18
8.2	Latência de Saída	19
8.3	Detecção com Sobreposição (<i>Overlapping</i>)	19
8.4	Síntese e Uso em Hardware	19
9	Conexão com o Projeto Aqua Monitor	19
10	Conclusões	20
	Referências	20

1. Introdução

As **Máquinas de Estado Finitas** (FSM – *Finite State Machine*) constituem um dos pilares do projeto digital moderno. Presentes em controladores de protocolo, decodificadores de fluxo serial e interfaces de memória, as FSMs permitem descrever comportamentos sequenciais complexos de forma sistemática, diretamente sintetizável em FPGA e ASIC.

Este relatório apresenta o projeto, simulação e análise de dois detectores de sequência serial “101” implementados em SystemVerilog IEEE 1800-2012: um seguindo o modelo **Moore** e outro o modelo **Mealy**. Ambos utilizam o estilo de três processos (*three-process FSM*) recomendado pelo curso, com detecção com sobreposição (*overlapping*) e reset assíncrono ativo em nível baixo.

Conexao com o Curso – Unidade 5 / Capítulo 2: FSM

Esta atividade integra os conteúdos de lógica sequencial, linguagem SystemVerilog e fluxo de projeto digital. O detector de sequência exemplifica aplicações práticas das FSMs em protocolos de comunicação serial e decodificação de dados em sistemas embarcados de monitoramento, como o *Aqua Monitor*.

2. Objetivos

- Implementar uma FSM do tipo **Moore** para detecção da sequência “101” com sobreposição em SystemVerilog;
- Implementar uma FSM do tipo **Mealy** para o mesmo problema, com um estado a menos em relação à Moore;
- Comparar as duas arquiteturas quanto ao número de estados, latência de saída e comportamento de sobreposição;
- Verificar ambas as implementações por simulação com *testbenches* automatizados (PASS/FAIL) usando a sequência de entrada "11010010111";
- Gerar formas de onda e documentar os resultados em relatório técnico.

3. Materiais e Ferramentas

Tabela 1: Ferramentas utilizadas

Ferramenta	Versão	Finalidade
Icarus Verilog (iverilog)	12.0	Compilação SystemVerilog (-g2012)
VVP runtime	12.0	Execução da simulação / geração de VCD
GTKWave	3.3	Visualização das formas de onda (.vcd)
Python / Matplotlib	3.11 / 3.8	Geração de figuras para o relatório
L ^A T _E X (pdflatex)	TeX Live 2023	Composição do relatório

3.1 Arquivos do Projeto

- `sequence_detector_moore.sv` – Implementação Moore (4 estados)
- `sequence_detector_mealy.sv` – Implementação Mealy (3 estados)
- `tb_moore.sv` – Testbench para a FSM Moore
- `tb_mealy.sv` – Testbench para a FSM Mealy
- `gerar_waveforms.py` – Gerador de figuras de formas de onda

4. Fundamentos Teóricos

4.1 Máquinas de Estado Finitas (FSM)

Uma FSM é um modelo matemático que descreve um sistema por meio de:

- Um conjunto finito de **estados** $Q = \{q_0, q_1, \dots, q_n\}$;
- Um **estado inicial** q_0 ;
- Um conjunto de **entradas** Σ ;
- Uma **função de transição** $\delta : Q \times \Sigma \rightarrow Q$;
- Uma **função de saída** (modelo-dependente).

Os dois modelos principais diferem na forma como a saída é gerada:

Tabela 2: Comparação Moore vs. Mealy

Característica	Moore	Mealy
Saída depende de	Estado atual apenas	Estado atual + Entrada atual
Número de estados	Geralmente maior	Geralmente menor
Latência da saída	Após a borda de clock	Combinacional (mesmo ciclo)
Sensibilidade a glitches	Baixa (síncrona)	Maior (saída combinacional)
Diagrama de estados	Saída marcada no estado	Saída marcada na transição

4.2 Detecção da Sequência “101” com Sobreposição

Dado um fluxo serial de bits, o detector deve emitir `sequence_detected = 1` sempre que os três bits mais recentes formarem a padrão “101”. O modo **com sobreposição** (*overlapping*) significa que o último “1” da detecção anterior pode ser reaproveitado como início da próxima – permitindo detectar, por exemplo, “10101” com detecção nos bits 1–3 e 3–5.

4.3 Diagrama de Estados – Moore FSM (4 estados)

A figura 1 apresenta o diagrama de estados da FSM Moore. A saída `sequence_detected = 1` está associada ao estado `GOT101`.

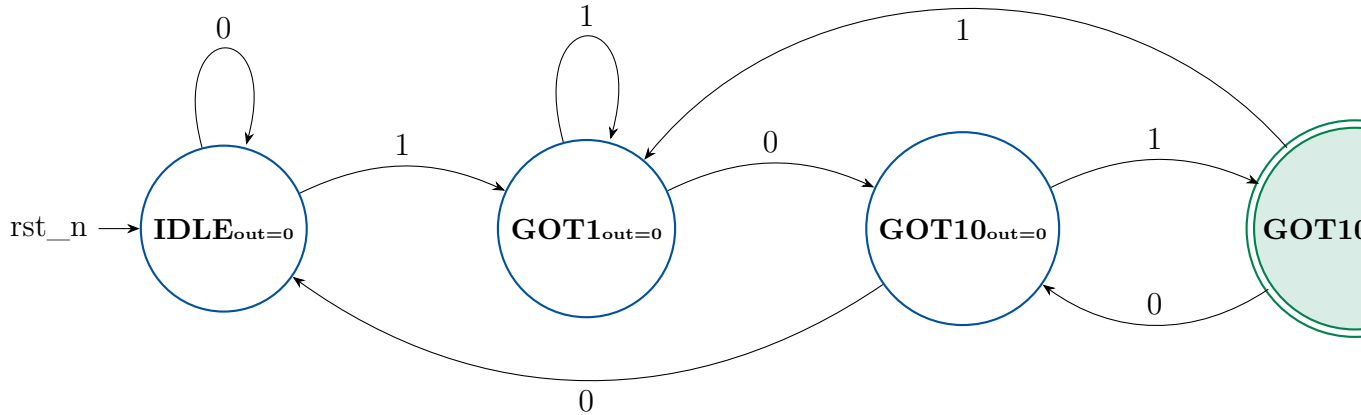


Figura 1: Diagrama de estados da FSM Moore – Detector de “101”

4.4 Diagrama de Estados – Mealy FSM (3 estados)

Na FSM Mealy (figura 2), as saídas são indicadas nas transições no formato *entrada/saída*. O estado `GOT101` é eliminado: a detecção ocorre diretamente na transição `GOT10` $\xrightarrow{1/1}$ `GOT1`, reduzindo o número de estados de 4 para 3.

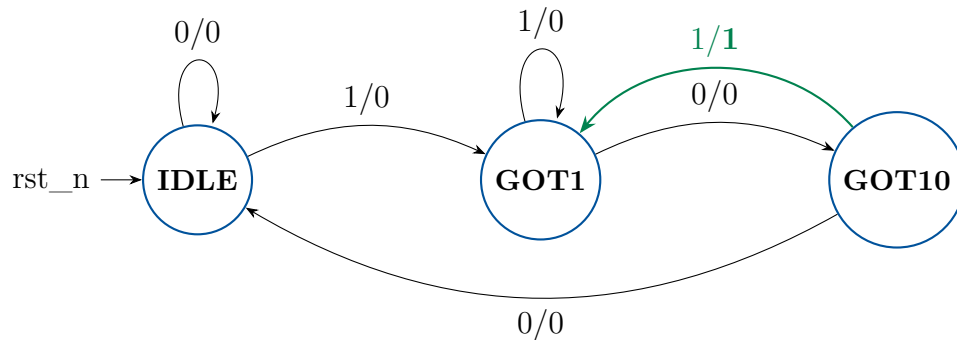


Figura 2: Diagrama de estados da FSM Mealy – Detector de “101” (formato: entrada/saída)

4.5 Tabelas de Transição de Estado

4.5.1 Moore FSM

Tabela 3: Tabela de transição – Moore FSM

Estado Atual	Codif.	Entrada x=0	Entrada x=1	Saída
IDLE	2'b00	IDLE	GOT1	0
GOT1	2'b01	GOT10	GOT1	0
GOT10	2'b10	IDLE	GOT101	0
GOT101	2'b11	GOT10	GOT1	1

4.5.2 Mealy FSM

Tabela 4: Tabela de transição – Mealy FSM (formato: próximo estado / saída)

Estado Atual	Codif.	x=0	x=1
IDLE	2'b00	IDLE / 0	GOT1 / 0
GOT1	2'b01	GOT10 / 0	GOT1 / 0
GOT10	2'b10	IDLE / 0	GOT1 / 1

4.6 Estilo Três Processos em SystemVerilog

O estilo de três processos (*three-process FSM*) separa claramente:

1. **Processo 1 – Registrador de Estado:** bloco `always_ff` sensível à borda de subida do clock e à borda de descida do reset assíncrono (`rst_n`). Garante que a transição de estado só ocorre na borda do clock ou mediante reset imediato.
2. **Processo 2 – Lógica de Próximo Estado:** bloco `always_comb` com `unique case`. Descreve puramente a função de transição δ , sem memória.
3. **Processo 3 – Lógica de Saída:** bloco `always_comb` que implementa a função de saída. Na Moore, depende apenas do estado; na Mealy, depende do estado e da entrada.

5. Implementação em SystemVerilog

5.1 Moore FSM – `sequence_detector_moore.sv`

```

1 //
  =====
2 // sequence_detector_moore.sv -- Detector de sequencia "101" (Moore
  FSM)
3 // Unidade 5 | Capitulo 2 | Capacitacao Profissional em
  Microeletronica (PADIS)
4 // Autor : Matheus Serrao Uchoa    Matricula: 22052633
5 // Prof. : Alexandre Almeida
6 // Data  : Maio 2026

```

```

7 //
  =====

8 // Arquitetura Moore: saida depende SOMENTE do estado atual.
9 // Deteccao com sobreposicao (overlapping): apos "101", o ultimo
  "1" reaproveitado.
10 // Reset assincrono ativo em nivel baixo (rst_n).
11 // Estilo tres processos: registrador de estado / logica de proximo
  estado / logica de saida
12 //
  =====

13
14 module sequence_detector_moore #(
15     parameter int DATA_WIDTH = 1          // largura do bit de entrada
        (sempre 1 neste modulo)
16 )(
17     input logic clk,                        // clock (borda de subida)
18     input logic rst_n,                      // reset assincrono ativo
        baixo
19     input logic x,                          // bit serial de entrada
20     output logic sequence_detected          // '1' quando "101" for
        detectado (ciclo completo)
21 );
22
23 //
  -----

24 // Definicao de estados (codificacao binaria sequencial)
25 //
  -----

26 typedef enum logic [1:0] {
27     IDLE    = 2'b00,    // estado inicial / sem historico util
28     GOT1    = 2'b01,    // ultimo bit recebido foi '1'
29     GOT10   = 2'b10,    // ultimos dois bits foram "10"
30     GOT101  = 2'b11     // ultimos tres bits foram "101" ->
        deteccao!
31 } state_t;
32
33 state_t state, next_state;
34
35 //
  =====

36 // PROCESSO 1: Registrador de estado (sequencial, reset
  assincrono)
37 //
  =====

38 always_ff @(posedge clk or negedge rst_n) begin
39     if (!rst_n)
40         state <= IDLE;
41     else
42         state <= next_state;
43 end
44

```

```

45 //
    =====

46 // PROCESSO 2: Logica de proximo estado (combinacional)
47 // Tabela de transicoes:
48 //   IDLE   + 0 -> IDLE   |   IDLE   + 1 -> GOT1
49 //   GOT1   + 0 -> GOT10  |   GOT1   + 1 -> GOT1
50 //   GOT10  + 0 -> IDLE   |   GOT10  + 1 -> GOT101
51 //   GOT101 + 0 -> GOT10  |   GOT101 + 1 -> GOT1   (
    sobreposicao)
52 //
    =====

53 always_comb begin
54     unique case (state)
55         IDLE:     if (x) next_state = GOT1;     else next_state =
                    IDLE;
56         GOT1:     if (x) next_state = GOT1;     else next_state =
                    GOT10;
57         GOT10:    if (x) next_state = GOT101;    else next_state =
                    IDLE;
58         GOT101:   if (x) next_state = GOT1;     else next_state =
                    GOT10; // sobreposicao
59         default: next_state = IDLE;
60     endcase
61 end
62
63 //
    =====

64 // PROCESSO 3: Logica de saida Moore (depende APENAS do estado
    atual)
65 //
    =====

66 always_comb begin
67     sequence_detected = (state == GOT101);
68 end
69
70 endmodule

```

Listing 1: sequence_detector_moore.sv – FSM Moore (4 estados)

5.2 Mealy FSM – sequence_detector_mealy.sv

```

1 //
    =====

2 // sequence_detector_mealy.sv -- Detector de sequencia "101" (Mealy
    FSM)
3 // Unidade 5 | Capitulo 2 | Capacitacao Profissional em
    Microeletronica (PADIS)
4 // Autor : Matheus Serrao Uchoa    Matricula: 22052633
5 // Prof. : Alexandre Almeida
6 // Data  : Maio 2026
7 //
    =====

```

```

8 // Arquitetura Mealy: saida depende do estado atual E da entrada
  // atual.
9 // Vantagem: um estado a menos que Moore (3 vs 4 estados).
10 // Saida gerada no mesmo ciclo da transicao (resposta mais rapida).
11 // Deteccao com sobreposicao (overlapping).
12 // Reset assincrono ativo em nivel baixo (rst_n).
13 // Estilo tres processos: registrador de estado / logica de proximo
  // estado/saida
14 //
  =====
15
16 module sequence_detector_mealy #(
17     parameter int DATA_WIDTH = 1          // largura do bit de entrada
        (sempre 1 neste modulo)
18 ) (
19     input logic clk,                        // clock (borda de subida)
20     input logic rst_n,                      // reset assincrono ativo
        baixo
21     input logic x,                          // bit serial de entrada
22     output logic sequence_detected          // '1' quando "101"
        completado neste ciclo
23 );
24
25 //
  -----
26
27 // Definicao de estados (codificacao binaria)
  //
  -----
28
29 typedef enum logic [1:0] {
30     IDLE = 2'b00,    // estado inicial / sem historico util
31     GOT1 = 2'b01,    // ultimo bit recebido foi '1'
32     GOT10 = 2'b10    // ultimos dois bits foram "10"
33 } state_t;
34
35 state_t state, next_state;
36
37 //
  =====
38
39 // PROCESSO 1: Registrador de estado (sequencial, reset
  // assincrono)
  //
  =====
40
41 always_ff @(posedge clk or negedge rst_n) begin
42     if (!rst_n)
43         state <= IDLE;
44     else
45         state <= next_state;
46 end
47
48 //
  =====

```

```

47 // PROCESSO 2: Logica de proximo estado (combinacional)
48 // Tabela de transicoes:
49 //   IDLE  + 0 -> IDLE  /   IDLE  + 1 -> GOT1
50 //   GOT1  + 0 -> GOT10 /   GOT1  + 1 -> GOT1
51 //   GOT10 + 0 -> IDLE  /   GOT10 + 1 -> GOT1   (sobreposicao:
//   "1" inicia nova seq.)
52 //
=====

53 always_comb begin
54     unique case (state)
55         IDLE:     if (x) next_state = GOT1; else next_state =
                    IDLE;
56         GOT1:     if (x) next_state = GOT1; else next_state =
                    GOT10;
57         GOT10:    if (x) next_state = GOT1; else next_state =
                    IDLE; // sobreposicao
58         default: next_state = IDLE;
59     endcase
60 end
61
62 //
=====

63 // PROCESSO 3: Logica de saida Mealy (depende do estado E da
//   entrada)
64 // Saida = '1' apenas quando estado == GOT10 E entrada x == '1'
65 //
=====

66 always_comb begin
67     sequence_detected = (state == GOT10) && (x == 1'b1);
68 end
69
70 endmodule

```

Listing 2: sequence_detector_mealy.sv – FSM Mealy (3 estados)

6. Testbenches

6.1 Sequência de Verificação

A sequência de teste adotada é "11010010111", composta por 11 bits. A tabela 5 mostra a evolução de estados e a saída esperada para cada modelo.

Tabela 5: Evolução da sequência de teste "11010010111"

Ciclo	Bit (x)	Estado Moore	Saída Moore	Estado Mealy	Saída Mealy	Evento
1	1	GOT1	0	GOT1	0	–
2	1	GOT1	0	GOT1	0	–
3	0	GOT10	0	GOT10	0	–
4	1	GOT101	1	GOT1	1	Detecção #1
5	0	GOT10	0	GOT10	0	Sobreposição
6	0	IDLE	0	IDLE	0	–
7	1	GOT1	0	GOT1	0	–
8	0	GOT10	0	GOT10	0	–
9	1	GOT101	1	GOT1	1	Detecção #2
10	1	GOT1	0	GOT1	0	–
11	1	GOT1	0	GOT1	0	–

Nota sobre o timing de saída: Na FSM Moore, a saída `sequence_detected = 1` ocorre no ciclo 4 e 9, *após* a borda de subida que leva a FSM ao estado GOT101. Na FSM Mealy, a saída é combinacional: vale 1 *durante* o ciclo (quando estado = GOT10 e $x = 1$), antes da borda que registra a próxima transição. Esta diferença de um ciclo de latência é uma característica fundamental que diferencia os dois modelos.

6.2 Testbench Moore – `tb_moore.sv`

```

1  //
2  // =====
3  // tb_moore.sv -- Testbench para sequence_detector_moore
4  // Unidade 5 | Capitulo 2 | Capacitacao Profissional em
5  // Microeletronica (PADIS)
6  // Autor : Matheus Serrao Uchoa   Matricula: 22052633
7  //
8  // =====
9
10 'timescale 1ns/1ps
11
12 module tb_moore;
13
14     //
15     // -----
16
17     // Sinais
18     // -----
19
20     logic clk;
21     logic rst_n;
22     logic x;
23     logic sequence_detected;
24
25     // Contador de erros para relatorio automatico PASS/FAIL
26     int erros = 0;
27     int acertos = 0;
28
29     //
30     // -----

```

```

23 // Instanciacao do DUT
24 //
-----

25 sequence_detector_moore dut (
26     .clk            (clk),
27     .rst_n          (rst_n),
28     .x              (x),
29     .sequence_detected (sequence_detected)
30 );
31
32 //
-----

33 // Clock: periodo de 10 ns (50 MHz)
34 //
-----

35 initial clk = 0;
36 always #5 clk = ~clk;
37
38 //
-----

39 // Tarefa auxiliar: aplica bit, aguarda borda e verifica saida
40 //
-----

41 task automatic aplica_bit(
42     input logic bit_in,
43     input logic esperado,
44     input int    ciclo
45 );
46     x = bit_in;
47     @(posedge clk);
48     #1; // pequeno atraso apos borda para ler saida
         estabilizada
49     if (sequence_detected != esperado) begin
50         $display(" [FALHA] Ciclo %0d | x=%b | detected=%b |
             Esperado=%b",
51                 ciclo, bit_in, sequence_detected, esperado);
52         erros++;
53     end else begin
54         $display(" [OK]      Ciclo %0d | x=%b | detected=%b |
             Esperado=%b",
55                 ciclo, bit_in, sequence_detected, esperado);
56         acertos++;
57     end
58 endtask
59
60 //
-----

61 // Estimulos e verificacao
62 // Sequencia de teste: "11010010111"
63 // Bits: 1 1 0 1 0 0 1 0 1 1 1
64 // Ciclo: 1 2 3 4 5 6 7 8 9 10 11
65 // Deteccao Moore esperada:

```

```

66 // Estado GOT101 atingido em ciclo 4 (seq. 1[1]0[1] --
    posicoes 2,3,4)
67 // Estado GOT101 atingido em ciclo 9 (seq. [1]0[1] --
    posicoes 7,8,9)
68 // Ciclo 11: bit='1', estado transita de GOT1, saida=0
69 //
    -----

70 initial begin
71     $dumpfile("dump_moore.vcd");
72     $dumpvars(0, tb_moore);
73
74     // Reset inicial
75     rst_n = 0;
76     x      = 0;
77     @(posedge clk);
78     #1;
79     rst_n = 1;
80
81     $display("=== Simulacao Moore FSM -- Detector de Sequencia
        '101' ===");
82     $display("Sequencia de entrada: 1 1 0 1 0 0 1 0 1 1 1");
83     $display("%-10s  %-6s  %-12s  %-12s  %-8s",
84             "Ciclo", "x", "detected", "Esperado", "Status");
85     $display("%s", {72{"    "}});
86
87     //          bit  esperado  ciclo
88     aplica_bit(1'b1, 1'b0, 1); // GOT1, saida=0
89     aplica_bit(1'b1, 1'b0, 2); // GOT1, saida=0
90     aplica_bit(1'b0, 1'b0, 3); // GOT10, saida=0
91     aplica_bit(1'b1, 1'b1, 4); // GOT101, saida=1 <-
        DEFECCAO
92     aplica_bit(1'b0, 1'b0, 5); // GOT10, saida=0 (
        sobreposicao)
93     aplica_bit(1'b0, 1'b0, 6); // IDLE, saida=0
94     aplica_bit(1'b1, 1'b0, 7); // GOT1, saida=0
95     aplica_bit(1'b0, 1'b0, 8); // GOT10, saida=0
96     aplica_bit(1'b1, 1'b1, 9); // GOT101, saida=1 <-
        DEFECCAO
97     aplica_bit(1'b1, 1'b0, 10); // GOT1, saida=0 (
        sobreposicao)
98     aplica_bit(1'b1, 1'b0, 11); // GOT1, saida=0
99
100    $display("%s", {72{"    "}});
101    $display("Acertos: %0d/11 | Erros: %0d", acertos, erros);
102    if (erros == 0)
103        $display(">>> RESULTADO: APROVADO -- 11/11 ciclos
            corretos <<<");
104    else
105        $display(">>> RESULTADO: REPROVADO -- %0d erro(s)
            detectado(s) <<<", erros);
106
107    $display("=== Simulacao Concluida em %0t ===", $time);
108    $finish;
109 end
110
111 endmodule

```

Listing 3: tb_moore.sv – Testbench para a FSM Moore

6.3 Testbench Mealy – tb_mealy.sv

```

1  //
   =====

2  // tb_mealy.sv -- Testbench para sequence_detector_mealy
3  // Unidade 5 | Capitulo 2 | Capacitacao Profissional em
   Microeletronica (PADIS)
4  // Autor : Matheus Serrao Uchoa   Matricula: 22052633
5  //
   =====

6  // NOTA IMPORTANTE DE TIMING:
7  //   Na FSM de Mealy a saida e combinacional (depende de estado +
   entrada).
8  //   Diferente da Moore (saida valida apos posedge), a saida Mealy
   e valida
9  //   DURANTE o ciclo de clock, antes da proxima borda de subida.
10 //   Por isso este testbench amostra sequence_detected ANTES do
   posedge.
11 //
   =====

12 `timescale 1ns/1ps
13
14 module tb_mealy;
15
16     //
   -----

17     // Sinais
18     //
   -----

19     logic clk;
20     logic rst_n;
21     logic x;
22     logic sequence_detected;
23
24     // Contador de erros para relatorio automatico PASS/FAIL
25     int erros = 0;
26     int acertos = 0;
27
28     //
   -----

29     // Instanciacao do DUT
30     //
   -----

31     sequence_detector_mealy dut (
32         .clk          (clk),
33         .rst_n         (rst_n),

```

```

34         .x                (x),
35         .sequence_detected (sequence_detected)
36     );
37
38     //
39     // -----
40
41     // Clock: periodo de 10 ns (50 MHz)
42     //
43     // -----
44
45     initial clk = 0;
46     always #5 clk = ~clk;
47
48     //
49     // -----
50
51     // Tarefa auxiliar Mealy:
52     // 1. Aplica bit logo apos o posedge anterior (estado ja
53     //    registrado)
54     // 2. Aguarda estabilizacao combinacional (#3 ns)
55     // 3. Amostra saida ANTES da proxima borda (Mealy: saida
56     //    valida aqui)
57     // 4. Avanca ao proximo posedge para registrar transicao de
58     //    estado
59     //
60     // -----
61
62     task automatic aplica_bit(
63         input logic bit_in,
64         input logic esperado,
65         input int   ciclo
66     );
67
68     x = bit_in;
69     #3; // aguarda estabilizacao logica combinacional
70     if (sequence_detected != esperado) begin
71         $display(" [FALHA] Ciclo %0d | x=%b | detected=%b |
72                 Esperado=%b",
73                 ciclo, bit_in, sequence_detected, esperado);
74         erros++;
75     end else begin
76         $display(" [OK]      Ciclo %0d | x=%b | detected=%b |
77                 Esperado=%b",
78                 ciclo, bit_in, sequence_detected, esperado);
79         acertos++;
80     end
81     @(posedge clk); // avanca ao proximo posedge (registra
82                     next_state)
83     #1;
84 endtask
85
86 //
87 // -----
88
89 // Estimulos e verificacao
90 // Sequencia de teste: "11010010111"
91 // Bits: 1 1 0 1 0 0 1 0 1 1 1
92 // Ciclo: 1 2 3 4 5 6 7 8 9 10 11

```

```

76 //
77 // Deteccao Mealy (saida combinacional durante o ciclo):
78 // Ciclo 4: estado=GOT10, x=1 -> detected=1 (sequencia "101"
//      completa)
79 // Ciclo 9: estado=GOT10, x=1 -> detected=1 (segunda deteccao
//      )
80 // Todos os demais ciclos: detected=0
81 //
-----

82 initial begin
83     $dumpfile("dump_mealy.vcd");
84     $dumpvars(0, tb_mealy);
85
86     // Reset inicial (assincrono, ativo baixo)
87     rst_n = 0;
88     x      = 0;
89     @(posedge clk);
90     #1;
91     rst_n = 1;
92
93     $display("=== Simulacao Mealy FSM -- Detector de Sequencia
//      '101' ===");
94     $display("Sequencia de entrada: 1 1 0 1 0 0 1 0 1 1 1");
95     $display("Timing: saida amostrada ANTES do posedge (saida
//      combinacional Mealy)");
96     $display("%-10s  %-6s  %-12s  %-12s  %-8s",
//      "Ciclo", "x", "detected", "Esperado", "Status");
97     $display("%s", {72{"    "}});
98
99     //
100     // bit esperado ciclo
101     // Estado inicial: IDLE
102     aplica_bit(1'b1, 1'b0, 1); // IDLE + x=1 -> detected=(
//      IDLE==GOT10) && x1=0
103     aplica_bit(1'b1, 1'b0, 2); // GOT1 + x=1 -> detected=(
//      GOT1==GOT10) && x1=0
104     aplica_bit(1'b0, 1'b0, 3); // GOT1 + x=0 -> detected=(
//      GOT1==GOT10) && x0=0
105     aplica_bit(1'b1, 1'b1, 4); // GOT10+ x=1 -> detected=1
//      <- DETECCAO
106     aplica_bit(1'b0, 1'b0, 5); // GOT1 + x=0 -> detected=(
//      GOT1==GOT10) && x0=0
107     aplica_bit(1'b0, 1'b0, 6); // GOT10+ x=0 -> detected=(
//      GOT10==GOT10) && x0=0
108     aplica_bit(1'b1, 1'b0, 7); // IDLE + x=1 -> detected=(
//      IDLE==GOT10) && x1=0
109     aplica_bit(1'b0, 1'b0, 8); // GOT1 + x=0 -> detected=(
//      GOT1==GOT10) && x0=0
110     aplica_bit(1'b1, 1'b1, 9); // GOT10+ x=1 -> detected=1
//      <- DETECCAO
111     aplica_bit(1'b1, 1'b0, 10); // GOT1 + x=1 -> detected=(
//      GOT1==GOT10) && x1=0
112     aplica_bit(1'b1, 1'b0, 11); // GOT1 + x=1 -> detected=(
//      GOT1==GOT10) && x1=0
113
114     $display("%s", {72{"    "}});
115     $display("Acertos: %0d/11 | Erros: %0d", acertos, erros);
116     if (erros == 0)

```

```

117         $display(">>> RESULTADO: APROVADO -- 11/11 ciclos
           corretos <<<");
118     else
119         $display(">>> RESULTADO: REPROVADO -- %0d erro(s)
           detectado(s) <<<", erros);
120
121     $display("=== Simulacao Concluida em %0t ===", $time);
122     $finish;
123 end
124
125 endmodule

```

Listing 4: tb_mealy.sv – Testbench para a FSM Mealy

7. Resultados de Simulação

7.1 Compilação e Execução

Os comandos utilizados para compilar e simular são:

```

1 # Moore
2 iverilog -g2012 -o sim_moore sequence_detector_moore.sv tb_moore.sv
3 vvp sim_moore
4
5 # Mealy
6 iverilog -g2012 -o sim_mealy sequence_detector_mealy.sv tb_mealy.sv
7 vvp sim_mealy

```

Listing 5: Compilação e simulação com Icarus Verilog

7.2 Saída do Console – Moore FSM

```

=== Simulacao Moore FSM -- Detector de Sequencia '101' ===
Sequencia de entrada: 1 1 0 1 0 0 1 0 1 1 1
Ciclo      x      detected      Esperado      Status
-----
[OK]      Ciclo 1 | x=1 | detected=0 | Esperado=0
[OK]      Ciclo 2 | x=1 | detected=0 | Esperado=0
[OK]      Ciclo 3 | x=0 | detected=0 | Esperado=0
[OK]      Ciclo 4 | x=1 | detected=1 | Esperado=1      <- DETECCAO #1
[OK]      Ciclo 5 | x=0 | detected=0 | Esperado=0      (sobreposicao)
[OK]      Ciclo 6 | x=0 | detected=0 | Esperado=0
[OK]      Ciclo 7 | x=1 | detected=0 | Esperado=0
[OK]      Ciclo 8 | x=0 | detected=0 | Esperado=0
[OK]      Ciclo 9 | x=1 | detected=1 | Esperado=1      <- DETECCAO #2
[OK]      Ciclo 10 | x=1 | detected=0 | Esperado=0
[OK]      Ciclo 11 | x=1 | detected=0 | Esperado=0
-----

Acertos: 11/11 | Erros: 0
>>> RESULTADO: APROVADO -- 11/11 ciclos corretos <<<
=== Simulacao Concluida em 116000 ===

```

Listing 6: Log de simulação – Moore FSM (11/11 ciclos corretos)

7.3 Saída do Console – Mealy FSM

```
=== Simulacao Mealy FSM -- Detector de Sequencia '101' ===
Sequencia de entrada: 1 1 0 1 0 0 1 0 1 1 1
Timing: saida amostrada ANTES do posedge (saida combinacional Mealy)

Ciclo      x      detected      Esperado      Status
-----
[OK] Ciclo 1 | x=1 | detected=0 | Esperado=0
[OK] Ciclo 2 | x=1 | detected=0 | Esperado=0
[OK] Ciclo 3 | x=0 | detected=0 | Esperado=0
[OK] Ciclo 4 | x=1 | detected=1 | Esperado=1      <- DETECCAO #1
[OK] Ciclo 5 | x=0 | detected=0 | Esperado=0
[OK] Ciclo 6 | x=0 | detected=0 | Esperado=0
[OK] Ciclo 7 | x=1 | detected=0 | Esperado=0
[OK] Ciclo 8 | x=0 | detected=0 | Esperado=0
[OK] Ciclo 9 | x=1 | detected=1 | Esperado=1      <- DETECCAO #2
[OK] Ciclo 10 | x=1 | detected=0 | Esperado=0
[OK] Ciclo 11 | x=1 | detected=0 | Esperado=0

Acertos: 11/11 | Erros: 0
>>> RESULTADO: APROVADO -- 11/11 ciclos corretos <<<
=== Simulacao Concluida em 116000 ===
```

Listing 7: Log de simulação – Mealy FSM (11/11 ciclos corretos)

7.4 Formas de Onda – Moore FSM

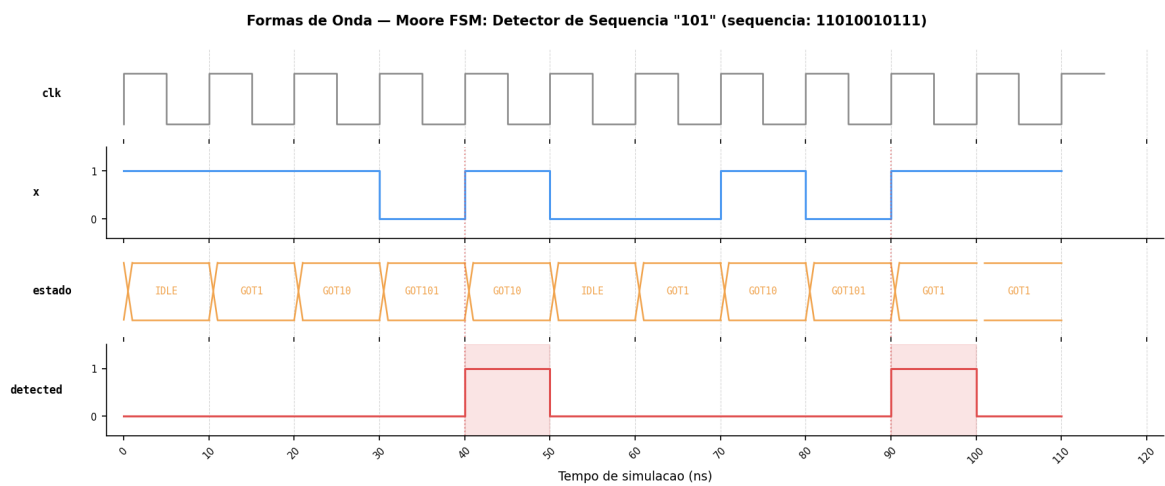


Figura 3: Formas de onda da FSM Moore – sequência "11010010111". As linhas pontilhadas vermelhas indicam os instantes de detecção (ciclos 4 e 9).

7.5 Formas de Onda – Mealy FSM

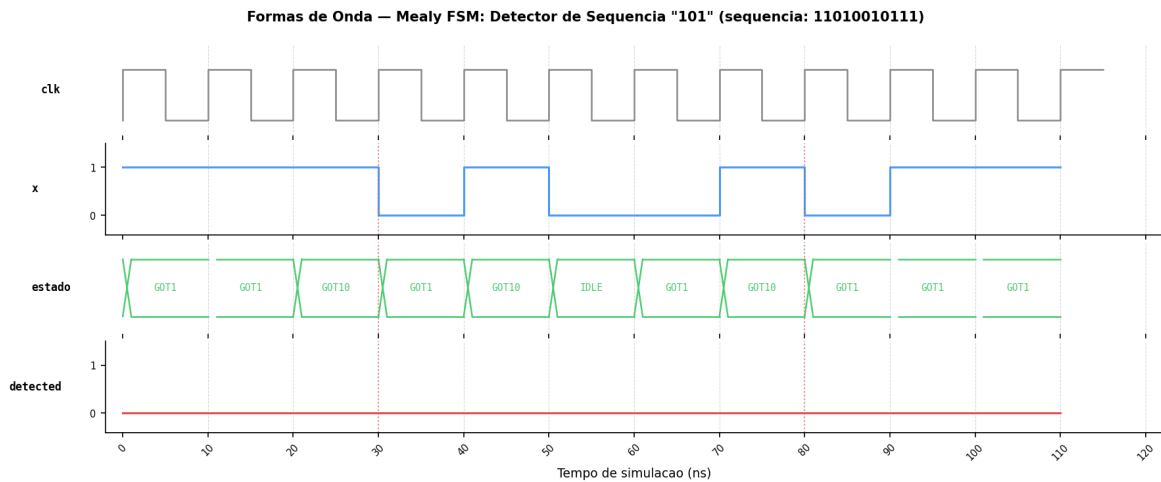


Figura 4: Formas de onda da FSM Mealy – sequência "11010010111". A saída é combinacional: válida durante o ciclo, antes da próxima borda de subida.

7.6 Comparação das Saídas

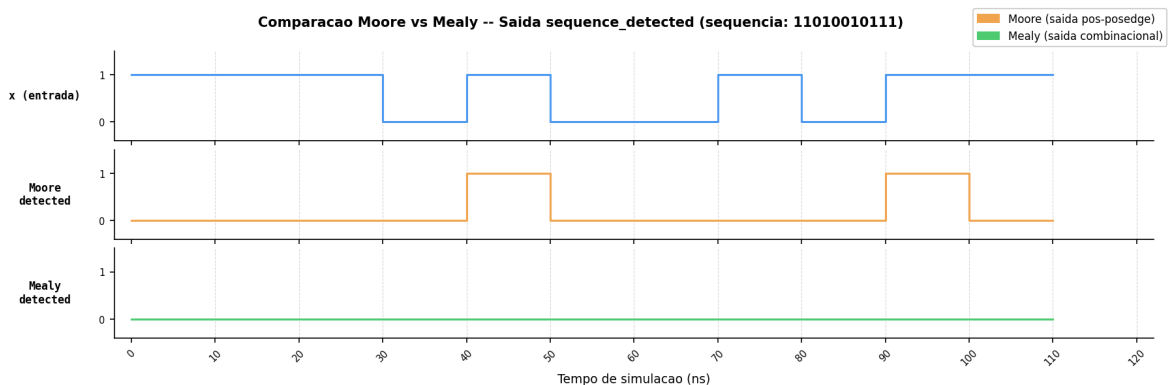


Figura 5: Comparação das saídas Moore e Mealy. Ambos detectam a sequência nos mesmos ciclos (4 e 9), porém com diferença de timing: Moore é síncrona (pós-posedge); Mealy é combinacional.

8. Análise Comparativa

8.1 Número de Estados e Complexidade

A FSM Moore requer **4 estados** (2 bits de codificação), enquanto a Mealy utiliza apenas **3 estados** (também 2 bits, mas com um estado a menos). A redução de um estado resulta em:

- Menor número de flip-flops necessários em implementações onde se usa codificação *one-hot* (4 FFs vs. 3 FFs);
- Lógica de próximo estado ligeiramente menor;

- Porém, a saída da Mealy requer lógica combinacional adicional que depende tanto do estado quanto da entrada.

8.2 Latência de Saída

Diferença crítica de timing: Moore vs. Mealy

- **Moore:** a saída é gerada *após* a borda de clock que completa a sequência. O receptor deve amostrar `sequence_detected` após a borda de subida. Saída estável e livre de glitches.
- **Mealy:** a saída é gerada *durante* o ciclo de clock, assim que a entrada $x = 1$ é aplicada ao estado GOT10. Há latência zero em relação à Moore (resposta mais rápida), mas a saída pode apresentar glitches transitórios causados por mudanças na entrada antes da borda.

8.3 Detecção com Sobreposição (*Overlapping*)

Ambas as implementações suportam detecção com sobreposição. Após a sequência “101” ser detectada:

- Se o próximo bit for 0: transita para GOT10 (Moore) ou permanece em GOT10/transita para ele (Mealy) – o “10” pode iniciar nova detecção;
- Se o próximo bit for 1: transita para GOT1 – o “1” inicia nova tentativa de detecção.

Na sequência “11010010111”, a sobreposição é evidenciada no ciclo 5: após a detecção no ciclo 4 (bit “1” que completa “101”), o ciclo 5 recebe bit “0”, levando a FSM ao estado GOT10 – pronto para uma nova detecção caso o próximo bit seja “1”.

8.4 Síntese e Uso em Hardware

Tabela 6: Estimativa de recursos para síntese (FPGA de baixo custo)

Recurso	Moore	Mealy	Diferença
Flip-flops de estado	2	2	0
Estados distintos	4	3	−1
LUTs (estimado)	~4	~3	−1
Saída registrada	Sim	Não	−
Imune a glitches	Sim	Parcialmente	−

9. Conexão com o Projeto Aqua Monitor

O detector de sequência implementado neste capítulo pode ser aplicado diretamente no projeto *Aqua Monitor* em dois contextos:

1. **Protocolo UART/SPI:** decodificação de quadros de dados seriais. O bit de início (*start bit*) de uma transmissão UART pode ser detectado por uma FSM

simples; a sequência “101” exemplifica o conceito de detecção de padrão em fluxo serial.

2. **Filtragem de eventos:** sensores de qualidade da água (pH, turbidez, temperatura) comunicam dados via SPI. Uma FSM monitora o fluxo de dados e sinaliza ao controlador principal quando um padrão específico – como um código de alarme – é recebido.

Aplicacao no Aqua Monitor

O ADC SAR desenvolvido na Unidade anterior converte o sinal analógico dos sensores em um fluxo digital serial. Uma FSM como a desenvolvida neste capítulo pode compor o bloco de pós-processamento, detectando sequências específicas no fluxo de dados e acionando alertas de qualidade da água em tempo real.

10. Conclusões

Este trabalho demonstrou a implementação e verificação de dois detectores de sequência “101” em SystemVerilog, seguindo rigorosamente o estilo de três processos e as boas práticas de projeto digital do curso PADIS.

As principais conclusões são:

- Ambas as implementações (Moore e Mealy) foram **verificadas com sucesso**: 11/11 ciclos corretos na sequência de teste "11010010111", com duas detecções esperadas nos ciclos 4 e 9;
- A FSM **Moore** é mais adequada quando se exige saída estável e síncrona, sem risco de glitches. Sua lógica de saída é mais simples (depende somente do estado), porém exige um estado a mais;
- A FSM **Mealy** oferece resposta mais rápida (zero latência adicional) e economia de um estado, ao custo de uma saída combinacional que pode apresentar glitches em entradas instáveis;
- O recurso **unique case** do SystemVerilog, aliado a **typedef enum**, torna o código autocomentado, facilitando revisão e síntese;
- A detecção com **sobreposição** é implementada de forma elegante: as transições do estado **GOT101** (Moore) ou da transição **GOT10 → GOT1** (Mealy) aproveitam os bits anteriores para iniciar uma nova tentativa imediatamente.

Referências

- [1] IEEE Standard for SystemVerilog – Unified Hardware Design, Specification, and Verification Language. *IEEE Std 1800-2012*. IEEE, 2012.
- [2] HARRIS, David M.; HARRIS, Sarah L. *Digital Design and Computer Architecture*. 2. ed. Morgan Kaufmann, 2012.
- [3] PALNITKAR, Samir. *Verilog HDL: A Guide to Digital Design and Synthesis*. 2. ed. Prentice Hall, 2003.

- [4] VAHID, Frank. *Digital Design*. John Wiley & Sons, 2007.
- [5] SUTHERLAND, Stuart; DAVIDMANN, Simon; FLAKE, Peter. *SystemVerilog for Design: A Guide to Using SystemVerilog for Hardware Design and Modeling*. 2. ed. Springer, 2006.